LearnPathIQ - Intelligent Personalized E-Learning Recommendations

LearnPathIQ helps learners find the right courses and the most useful next learning action. Learners describe their background, interests, topics and goals; the platform combines profile matching, learning behavior, and predictive models to rank courses with plain-language reasons. Interaction history continuously refines future suggestions.

1. Product overview

Accounts, learner profiles, course catalog, ranked recommendations with explanations, ordered learning paths, engagement insights, and a transparency/analytics view with real model metrics.

2. Problem being solved

Learners face too many course options and unclear next steps. LearnPathIQ narrows choices to a short ranked list matched to profile and behavior, plus one clear recommended action (review content, take a quiz, join a discussion, review resources, or continue the path).

3. Key features

- Profile-aware hybrid ranking with explanations
- Behavior-driven personalization loop
- Success-probability estimates with contributing features
- Personalized action scores (estimated benefit, observational)
- Learning path, insights, activity history
- Offline evaluation and fairness audit reporting

4. User workflow

Register -> complete learner profile -> view dashboard and recommendations -> open/start courses -> log quizzes, content reviews, discussions -> recommendations adapt -> follow the ordered learning path -> review insights.

5. System architecture

FastAPI + Jinja2 frontend, SQLAlchemy/SQLite persistence, recommender service, trained model bundle (scaler, selected predictor, T-learner action models), metrics JSON and plots served to the analytics page. Binds to 127.0.0.1:8000.

6. Technology stack

Python, FastAPI, Uvicorn, SQLAlchemy, SQLite, Pydantic, pandas, NumPy, scikit-learn, XGBoost, joblib, matplotlib, Jinja2.

7. Data pipeline

Dataset source: synthetic-oulad-like-fallback-v1. Weekly learner-state features (cum_clicks, recent_clicks, quiz_count, forum_count, completions, engagement, weekly_hours, skill_code), learner-level 80/10/10 split (train 21120, val 2640, test 2640). Categorical encoding, missing-value imputation, scaling; cached artifacts avoid reprocessing per request.

8. Recommendation engine

Weighted hybrid: TF-IDF cosine content similarity (30%), skill compatibility (15%), topic fit (15%), predicted success (12%), behavioral signals (10%), goal fit (8%), rating and engagement (10%). Scores computed dynamically; catalog metadata is static but rankings are not.

9. ML models

Logistic Regression, Random Forest, XGBoost trained to predict learner success from learner-state features. Best by validation AUC is promoted. Confusion matrices, ROC curves, calibration (Brier) and feature importance are generated from the local run.

10. Personalization mechanism

Events (view/open/start/quiz/content/forum/complete/dismiss/save) are stored in SQLite and shift behavioral boosts, engagement scores and state vectors, so rankings differ as history grows (verified live).

11. Adaptive action recommendation

One-vs-rest T-learner estimates uplift(state, action) = P(success|action) - P(success|no action) where data supports it, else a predictive fallback. Policy selects argmax estimated benefit. UI wording is conservative: estimated benefit / personalized action score / based on observed patterns.

12. Evaluation methodology

Held-out learner-level test set; AUC/F1 primary; Brier for calibration. Offline policy value via behavior-policy estimation (multinomial logistic), propensity clipping, IPS/SNIPS/DR on logged data. No live A/B testing claims.

13. Actual model metrics (this training run)

logistic_regression: AUC=0.6028 acc=0.5936 prec=0.5605 rec=0.329 F1=0.4146 Brier=0.2378 cm=[[1187, 298], [775, 380]]

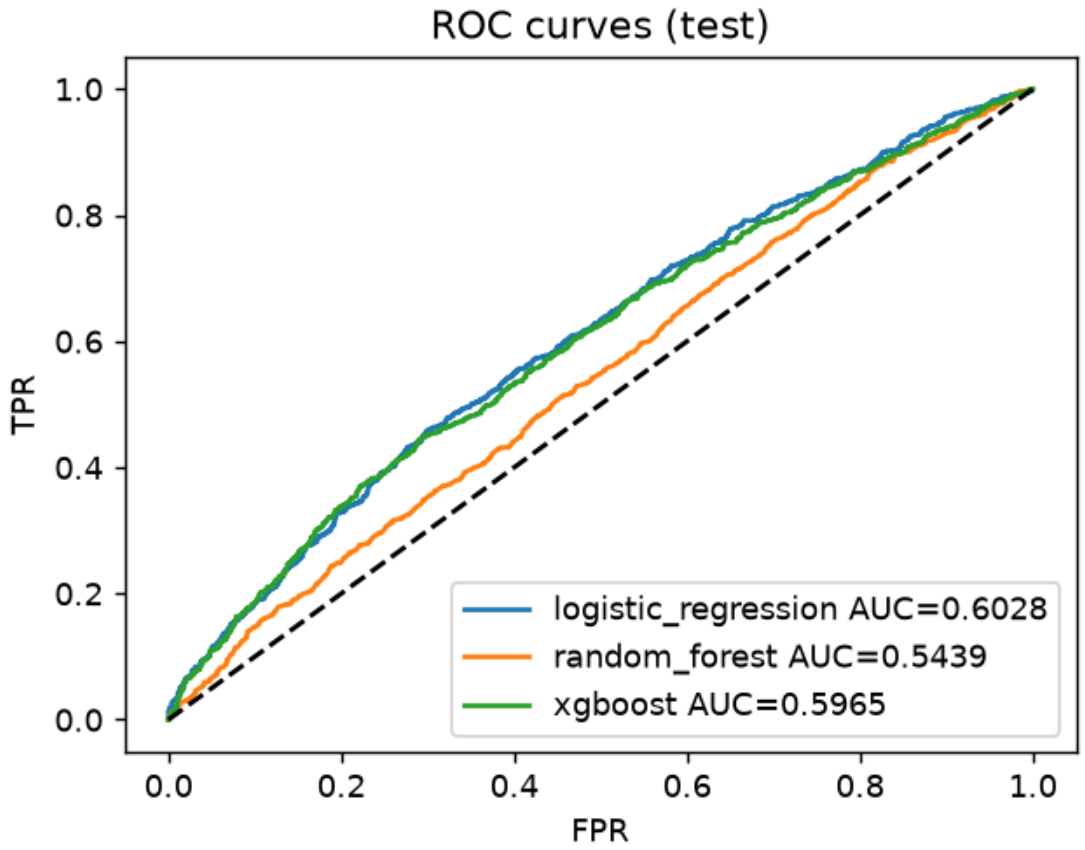
random_forest: AUC=0.5439 acc=0.5311 prec=0.4616 rec=0.432 F1=0.4463 Brier=0.2837 cm=[[903, 582], [656, 499]]

xgboost: AUC=0.5965 acc=0.5977 prec=0.5597 rec=0.3775 F1=0.4509 Brier=0.2405 cm=[[1142, 343], [719, 436]]

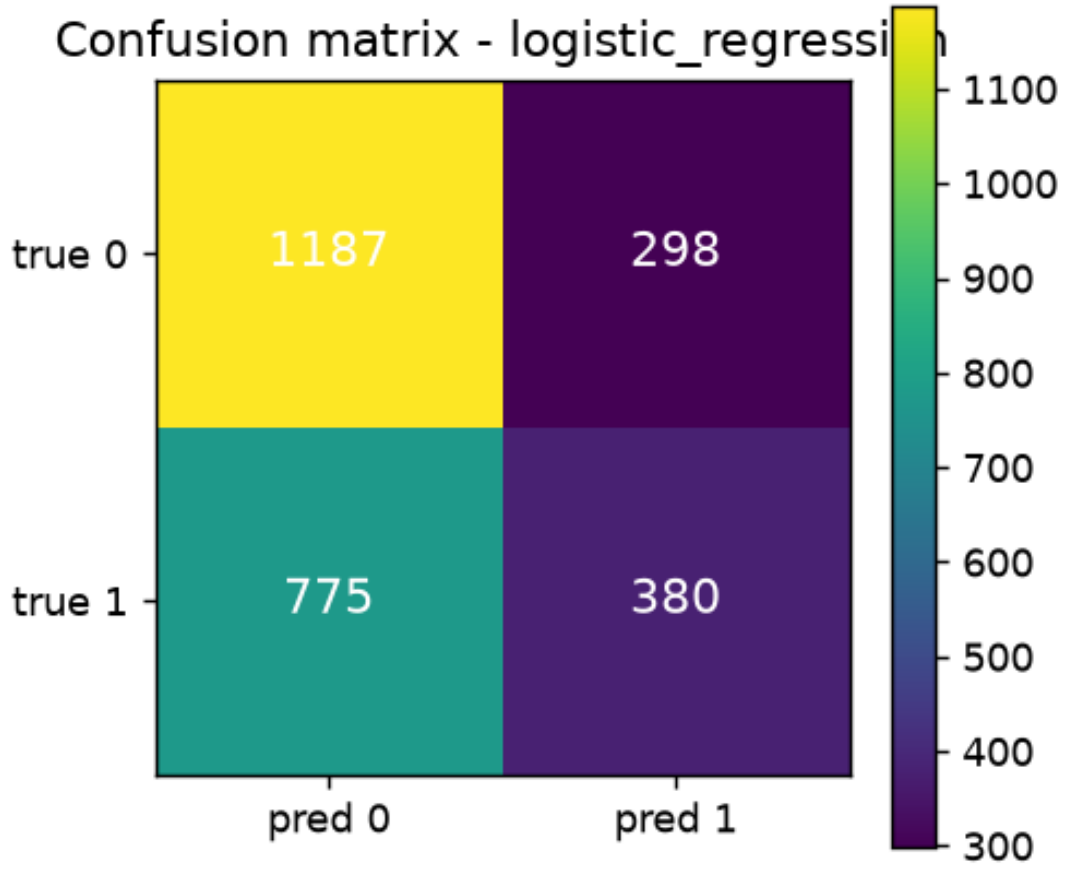
Selected model: logistic_regression. Offline policy: IPS=None SNIPS=None DR=None (unstable or unsupported: LogisticRegression.__init__() got an unexpected keyword argument 'multi_class').

14. Charts

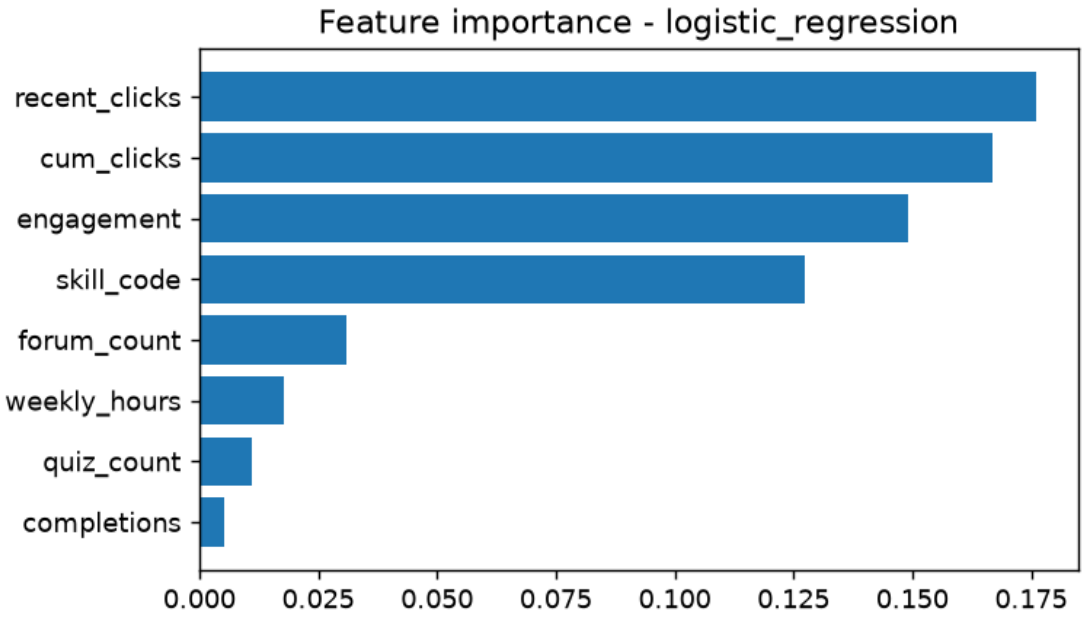
ROC curves:



Confusion matrix:



Feature importance:



15. Fairness audit


```
{
 "age_band": {
  "0-35": {
   "n": 1728,
   "success_rate": 0.445,
```

```
   "avg_predicted": 0.4364,
   "f1": 0.415
  },
  "35-55": {
   "n": 708,
   "success_rate": 0.4449,
   "avg_predicted": 0.4345,
   "f1": 0.4369
  },
  "55<=": {
   "n": 204,
   "success_rate": 0.348,
   "avg_predicted": 0.4153,
   "f1": 0.3048
  }
 },
 "gender": {
  "F": {
   "n": 1260,
   "success_rate": 0.4286,
   "avg_predicted": 0.4342,
   "f1": 0.4
  },
  "M": {
   "n": 1380,
   "success_rate": 0.4457,
   "avg_predicted": 0.4344,
   "f1": 0.4278
  }
 }
}
```


Audit-only reporting; no claim of perfect fairness.

16. Installation

Double-click run.bat (Windows), or: pip install -r requirements.txt. Optional: docker build -t learnpathiq .

17. Running instructions

run.bat starts http://localhost:8000 automatically. Manual: python scripts/train_pipeline.py, then python -m uvicorn app.main:app --host 127.0.0.1 --port 8000. Docker: docker run --rm -p 8000:8000 learnpathiq.

18. Limitations

Synthetic fallback data when the public dataset is unreachable; observational action scores are not causal proof; offline policy values are not live results; demo-grade auth; single-node SQLite.

19. Privacy/security

Local-first: all data in learnpathiq.db. Hashed passwords (pbkdf2_sha256), HttpOnly token cookie, validated inputs, parameterized queries, no secrets committed.